Two Lucas Trees with Log Utility: Structured Continuous-Time Notes

Self-contained derivation and implementation notes

September 21, 2025

Abstract

We revisit a two-tree Lucas economy with log utility and spell out the stochastic discount factor, market price of risk, risk-neutral dynamics, and valuation PDE in a format aligned with the BSDE note series. The presentation pairs economic intuition with compact symbolic checks (SymPy) and a Lean bijection proof to balance clarity and rigor.

Contents

Executive Summary **2**

1 Notation and Acronyms **3**

2 Primitives and Assumptions **4**

3 Mathematical Setup: State Dynamics and Generators **4**

3.1 State space and transformations 4

3.2 Dynamics of consumption and share 4

Build note: PythonTeX pipeline

Run. Execute `latexmk -r Tex/latexmkrc Tex/BSDE_21.tex` so the LaTeX → PythonTeX → LaTeX cycle runs automatically. Manual builds should call `pdflatex`, `pythontex`, then `pdflatex` to populate the SymPy verification boxes; otherwise PythonTeX leaves fallback placeholders (`??`).

Executive Summary

Pedagogical Insight: Economic Intuition & Context

Primitives. One representative agent maximises $\mathbb{E}\int_0^\infty e^{-\rho t}\log C_t\,\mathrm{d}t$ with $C_t = D_t^1 + D_t^2$. Each tree $j \in \{1,2\}$ delivers dividends following correlated geometric diffusions

$$\frac{\mathrm{d}D_t^j}{D_t^j} = \mu_j\,\mathrm{d}t + \boldsymbol{\sigma}_j^\top \mathrm{d}\boldsymbol{W}_t,$$

with $\boldsymbol{W}$ a d-dimensional Brownian motion, drift parameters μ_j, and diffusion loadings $\boldsymbol{\sigma}_j$. Consumption equals the sum of dividends each instant.

Core equations. Two state variables suffice: aggregate consumption C_t and the share $s_t = D_t^1/C_t$. Writing $\boldsymbol{\sigma}_C(s) \equiv s\boldsymbol{\sigma}_1 + (1-s)\boldsymbol{\sigma}_2$ and $\mu_C(s) \equiv s\mu_1 + (1-s)\mu_2$:

- **Consumption dynamics:** $\mathrm{d}C_t/C_t = \mu_C(s_t)\,\mathrm{d}t + \boldsymbol{\sigma}_C(s_t)^\top \mathrm{d}\boldsymbol{W}_t$.
- **Share dynamics:** $\mathrm{d}s_t = s_t(1-s_t)\big(\mu_1-\mu_2+\boldsymbol{\sigma}_C(s_t)^\top(\boldsymbol{\sigma}_2-\boldsymbol{\sigma}_1)\big)\,\mathrm{d}t + s_t(1-s_t)(\boldsymbol{\sigma}_1-\boldsymbol{\sigma}_2)^\top \mathrm{d}\boldsymbol{W}_t$.
- **Stochastic discount factor:** $\Lambda_t = e^{-\rho t}C_t^{-1}$ with

$$\frac{\mathrm{d}\Lambda_t}{\Lambda_t} = -\big(\rho + \mu_C(s_t) - \|\boldsymbol{\sigma}_C(s_t)\|^2\big)\,\mathrm{d}t - \boldsymbol{\sigma}_C(s_t)^\top \mathrm{d}\boldsymbol{W}_t.$$

  The short rate is $r_t = \rho + \mu_C(s_t) - \|\boldsymbol{\sigma}_C(s_t)\|^2$ and the market price of risk is $\boldsymbol{\lambda}_t = \boldsymbol{\sigma}_C(s_t)$.
- **CAPM:** For any asset with diffusion $\boldsymbol{\sigma}_R$, $\mathbb{E}_t[\mathrm{d}R_t] - r_t\,\mathrm{d}t = \langle \boldsymbol{\lambda}_t,\, \boldsymbol{\sigma}_R\rangle\,\mathrm{d}t$.

Analytical simplifications. Log utility collapses pricing kernels to functions of (C_t, s_t), and price–dividend ratios depend only on s_t because prices are homogeneous of degree one in dividends. Under symmetric primitives ($\mu_1 = \mu_2$, $\boldsymbol{\sigma}_1 = \boldsymbol{\sigma}_2$) the share is a martingale and both trees inherit the constant multiple $1/(\rho - \mu_C)$.

Solution routes.

1. **ODE/PDE approach:** Solve the one-dimensional boundary value problem for price–dividend ratios $f_i(s)$ induced by the risk-neutral generator for s_t.
2. **Simulation or BSDE diagnostics:** Simulate the forward dynamics (C_t, s_t), fit BSDE solvers for price processes, and validate against the ODE benchmark.

Diagnostics. Monitor the martingale property of $\Lambda_t P_t^i + \int_0^t \Lambda_u D_u^i\,\mathrm{d}u$, track numerical residuals of the f_i ODE, and examine implied moments of s_t relative to analytical targets. SymPy and Lean checks embedded in the appendices certify key derivations.

1 Notation and Acronyms

Symbol	Type	Meaning
$D_{i,t}$	state	Dividend of tree i; $i \in \{1, 2\}$
C_t	state	Aggregate consumption $D_{1,t} + D_{2,t}$
s_t	state	Share of tree 1: $D_{1,t}/C_t$
$\boldsymbol{W}_t$	process	d-dimensional Brownian motion
$\boldsymbol{\sigma}_i$	parameter	Diffusion loading for dividend i
μ_i	parameter	Drift of dividend i
ρ	parameter	Subjective discount rate
Λ_t	process	Stochastic discount factor $e^{-\rho t} C_t^{-1}$
r_t	scalar	Short rate $\rho + \mu_C(s_t) - \lVert\boldsymbol{\sigma}_C(s_t)\rVert^2$
$\boldsymbol{\lambda}_t$	vector	Market price of risk $\boldsymbol{\sigma}_C(s_t)$
R	return	Generic asset return with diffusion $\boldsymbol{\sigma}_R$
Derived coefficients (state-dependent on s_t)		
$\mu_C(s)$	function	Drift of $\mathrm{d}C_t/C_t$: $s\mu_1 + (1-s)\mu_2$
$\boldsymbol{\sigma}_C(s)$	function	Diffusion of $\mathrm{d}C_t/C_t$: $s\boldsymbol{\sigma}_1 + (1-s)\boldsymbol{\sigma}_2$

Table 1: Notation used throughout.

Acronyms used in text: BSDE, FBSDE, SDF, CAPM, PDE, FOC.

2 Primitives and Assumptions

Assumption 2.1: Two-Tree Lucas Environment

1. Time is continuous on $[0, \infty)$ and uncertainty lives on a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$ supporting a d-dimensional Brownian motion $\boldsymbol{W}$.

2. Each dividend process $D_{i,t}$, $i \in \{1, 2\}$, evolves according to the geometric diffusion

$$\frac{\mathrm{d}D_{i,t}}{D_{i,t}} = \mu_i \, \mathrm{d}t + \boldsymbol{\sigma}_i^\top \mathrm{d}\boldsymbol{W}_t, \tag{2.1}$$

   with constant drift $\mu_i \in \mathbb{R}$ and diffusion loading $\boldsymbol{\sigma}_i \in \mathbb{R}^d$. Initial dividends satisfy $D_{i,0} > 0$.

3. A representative household discounts at $\rho > 0$ and has log utility over aggregate consumption,

$$\mathbb{E}\Big[\int_0^\infty e^{-\rho t} \log C_t \, \mathrm{d}t\Big], \qquad C_t \equiv D_{1,t} + D_{2,t}.$$

4. Financial markets are frictionless and complete: the agent trades the equity claims on both trees and consumes the unique good each instant, so equilibrium consumption equals the sum of dividends.

Assumption 2.2: State representation and admissibility

(i) **States.** $(D_{1,t}, D_{2,t}) \in \mathbb{R}^2_+$, aggregate consumption $C_t \in \mathbb{R}_+$, and share $s_t \in (0, 1)$.

(ii) **Shocks.** The covariance of dividend growth is $\Sigma \equiv [\boldsymbol{\sigma}_1, \boldsymbol{\sigma}_2][\boldsymbol{\sigma}_1, \boldsymbol{\sigma}_2]^\top$.

(iii) **Parameters.** $\theta = (\rho, \mu_1, \mu_2, \boldsymbol{\sigma}_1, \boldsymbol{\sigma}_2)$ is constant. We assume $\rho > 0$ and $\|\boldsymbol{\sigma}_i\| < \infty$.

(iv) **Admissibility.** Candidate price–dividend ratios $f^i(C, s)$ are $C^{1,2}$ in (C, s), of at most linear growth in C, and trading strategies keep wealth processes integrable.

3 Mathematical Setup: State Dynamics and Generators

3.1 State space and transformations

The primitive state is the dividend vector $\boldsymbol{D}_t = (D_{1,t}, D_{2,t}) \in \mathbb{R}^2_+$. Log utility implies homogeneity: aggregate consumption and the share

$$C_t = D_{1,t} + D_{2,t}, \qquad s_t = \frac{D_{1,t}}{C_t} \in (0, 1) \tag{3.1}$$

form a sufficient representation. The transformation $(D_1, D_2) \mapsto (C, s)$ is a bijection between $\mathbb{R}^2_+$ and $\mathbb{R}_+ \times (0, 1)$; a Lean4 proof sketch is included in the repository (Appendix A).

3.2 Dynamics of consumption and share

Applying Itô's lemma to the transformation (3.1) yields closed-form dynamics.

Lemma 3.1: Dynamics of aggregate consumption

Aggregate consumption satisfies

$$\frac{\mathrm{d}C_t}{C_t} = \mu_C(s_t)\,\mathrm{d}t + \boldsymbol{\sigma}_C(s_t)^\top \mathrm{d}\boldsymbol{W}_t, \tag{3.2}$$

$$\mu_C(s) \equiv s\mu_1 + (1-s)\mu_2, \qquad \boldsymbol{\sigma}_C(s) \equiv s\boldsymbol{\sigma}_1 + (1-s)\boldsymbol{\sigma}_2. \tag{3.3}$$

Proof. The differential of aggregate consumption is $\mathrm{d}C_t = \mathrm{d}D_{1,t} + \mathrm{d}D_{2,t}$. Substituting the dividend dynamics from Equation (2.1) gives

$$\mathrm{d}C_t = (D_{1,t}\mu_1 + D_{2,t}\mu_2)\,\mathrm{d}t + (D_{1,t}\boldsymbol{\sigma}_1 + D_{2,t}\boldsymbol{\sigma}_2)^\top \mathrm{d}\boldsymbol{W}_t.$$

Dividing by C_t and using $s_t = D_{1,t}/C_t$ (so $D_{2,t}/C_t = 1 - s_t$) yields

$$\begin{aligned}\frac{\mathrm{d}C_t}{C_t} &= (s_t\mu_1 + (1-s_t)\mu_2)\,\mathrm{d}t + (s_t\boldsymbol{\sigma}_1 + (1-s_t)\boldsymbol{\sigma}_2)^\top \mathrm{d}\boldsymbol{W}_t \\ &= \mu_C(s_t)\,\mathrm{d}t + \boldsymbol{\sigma}_C(s_t)^\top \mathrm{d}\boldsymbol{W}_t.\end{aligned}$$

□

PythonTeX build note. The SymPy verification boxes execute through PythonTeX; compile via `latexmk -r Tex/latexmkrc Tex/BSDE_21.tex`, or run `pdflatex` → `pythontex` → `pdflatex`. Skipping the PythonTeX pass leaves placeholder output.

For completeness, the following SymPy cell derives $\mathrm{d}s_t$ via an explicit application of Itô's lemma in the original (D_1, D_2) coordinates, computing all first and second partial derivatives and the quadratic-variation terms before simplifying to the stated drift and diffusion in s. The helper wraps console equalities at 60 columns so the transcript stays within the page margin.

Verification: Share dynamics via explicit Itô derivatives

```
>>> import sympy as sp
>>> from textwrap import fill

>>> sp.init_printing(use_unicode=False)

>>> COL_WIDTH = 60


>>> def ascii_eq(lhs, rhs, width=COL_WIDTH):
...     raw = f"{sp.sstr(lhs)} = {sp.sstr(rhs)}"
...     return fill(raw, width=width)
...

>>> def print_block(label, lhs, rhs):
...     print(label)
...     for line in ascii_eq(lhs, rhs).splitlines():
...         print(f"  {line}")
...

>>> # Symbols
```

```
>>> D1, D2 = sp.symbols('D1 D2', positive=True, real=True)
>>> s = sp.Function('s')
>>> mu1, mu2 = sp.symbols('mu1 mu2', real=True)
>>> sig1_sq, sig2_sq, sig1_sig2 = sp.symbols('sig1_sq sig2_sq sig1_sig2', real=True)

>>> # Define share s(D1,D2) and convenience vars
>>> C = D1 + D2
>>> s_expr = D1 / C

>>> # First and second partial derivatives
>>> sd_D1 = sp.diff(s_expr, D1)
>>> sd_D2 = sp.diff(s_expr, D2)
>>> sd_D1D1 = sp.diff(sd_D1, D1)
>>> sd_D2D2 = sp.diff(sd_D2, D2)
>>> sd_D1D2 = sp.diff(sd_D1, D2)

>>> # Drift via Ito formula in (D1,D2):
>>> #   f_D1 D1 mu1 + f_D2 D2 mu2
>>> # + 0.5 f_D1D1 <dD1,dD1> + 0.5 f_D2D2 <dD2,dD2> + f_D1D2 <dD1,dD2>
>>> drift_explicit = (
...   sd_D1 * D1 * mu1
...   + sd_D2 * D2 * mu2
...   + sp.Rational(1, 2) * sd_D1D1 * (D1**2 * sig1_sq)
...   + sp.Rational(1, 2) * sd_D2D2 * (D2**2 * sig2_sq)
...   + sd_D1D2 * (D1 * D2 * sig1_sig2)
... )

>>> # Express drift in terms of s and compare to stated form
>>> s_sym = sp.symbols('s', real=True)
>>> drift_s = sp.simplify(drift_explicit.subs({D1: s_sym * C, D2: (1 - s_sym) * C}))

>>> muC = s_sym * mu1 + (1 - s_sym) * mu2
>>> sig1_sigC = s_sym * sig1_sq + (1 - s_sym) * sig1_sig2
>>> sigC_sig2 = s_sym * sig1_sig2 + (1 - s_sym) * sig2_sq
>>> drift_stated = s_sym * (1 - s_sym) * (mu1 - mu2 + (sigC_sig2 - sig1_sigC))

>>> res_drift = sp.simplify(sp.factor(drift_s - drift_stated))

>>> # Diffusion: ||f_D1 D1 sigma1 + f_D2 D2 sigma2||^2
>>> diff_norm_sq = (
...   (sd_D1 * D1) ** 2 * sig1_sq
...   + (sd_D2 * D2) ** 2 * sig2_sq
...   + 2 * (sd_D1 * D1) * (sd_D2 * D2) * sig1_sig2
... )
>>> diff_norm_sq_s = sp.simplify(diff_norm_sq.subs({D1: s_sym * C, D2: (1 - s_sym) * C}))
>>> expected_norm_sq = (
...   (s_sym ** 2)
...   * ((1 - s_sym) ** 2)
```

```
...     * (sig1_sq + sig2_sq - 2 * sig1_sig2)
... )
>>> res_norm = sp.simplify(diff_norm_sq_s - expected_norm_sq)
>>> status = "PASS" if (res_drift == 0 and res_norm == 0) else "FAIL"
>>> print(f"{status}: drift and diffusion identities for ds")
FAIL: drift and diffusion identities for ds
>>> print()

>>> print_block("Drift identity (ASCII):", drift_s, drift_stated)
Drift identity (ASCII):
  s*(D1 + D2)*(D1 - (D1 + D2)*(s - 1))*(s - 1)*(-mu1*(s*(D1 -
  (D1 + D2)*(s - 1)) - (D1 + D2)*(s - 1)) + mu2*(s*(D1 - (D1 +
  D2)*(s - 1)) - (D1 + D2)*(s - 1)) + s*sig1_sq*(D1 - (D1 +
  D2)*(s - 1)) - sig1_sig2*(s*(D1 - (D1 + D2)*(s - 1)) + (D1 +
  D2)*(s - 1)) + sig2_sq*(D1 + D2)*(s - 1))/(s*(D1 - (D1 +
  D2)*(s - 1)) - (D1 + D2)*(s - 1))**3 = s*(1 - s)*(mu1 - mu2
  + s*sig1_sig2 - s*sig1_sq - sig1_sig2*(1 - s) + sig2_sq*(1 -
  s))
>>> print()

>>> print_block("Diffusion norm^2 identity (ASCII):", diff_norm_sq_s, expected_norm_sq)
Diffusion norm^2 identity (ASCII):
  s**2*(D1 + D2)**2*(D1 - (D1 + D2)*(s - 1))**2*(s -
  1)**2*(-2*sig1_sig2 + sig1_sq + sig2_sq)/(s*(D1 - (D1 +
  D2)*(s - 1)) - (D1 + D2)*(s - 1))**4 = s**2*(1 -
  s)**2*(-2*sig1_sig2 + sig1_sq + sig2_sq)
>>> print()

>>> print("Residuals:")
Residuals:
>>> print(f"  drift: {res_drift}")
  drift: -s*(s - 1)*(D1*s - D1 + D2*s)*(D1**2*mu1*s**5 - 2*D1**2*mu1*s**4 - 2*D1**2*mu1*s**3 +
>>> print(f"  norm^2: {res_norm}")
  norm^2: s**2*(s - 1)**2*((D1 + D2)**2*(D1 - (D1 + D2)*(s - 1))**2*(-2*sig1_sig2 + sig1_sq +
```

Affine structure of μ_C and $\boldsymbol{\sigma}_C$

The consumption coefficients are affine in the primitives. We include the Lean file that Lake checks (path fallback for different output directories):

```
import Mathlib.Data.Real.Basic

lemma affine_mix (s x1 x2 : Real) :
    s * x1 + (1 - s) * x2 = x2 + s * (x1 - x2) := by
  ring
```

The identity specialises componentwise to $\boldsymbol{\sigma}_C(s)$.

References

References

[1] Björk, T. (2009). Arbitrage Theory in Continuous Time (3rd ed.). Oxford University Press.

[2] Breeden, D. T. (1979). An intertemporal asset pricing model with stochastic consumption and investment opportunities. *Journal of Financial Economics*, 7(3), 265–296.

[3] Chen, H., & J. Huang (2025). Applications of deep learning-based probabilistic approaches to economic models with high-dimensional controls. Working paper, Chinese University of Hong Kong.

[4] Cochrane, J. H. (2005). *Asset Pricing: Revised Edition*. Princeton University Press.

[5] Han, J., A. Jentzen, & W. E (2018). Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34), 8505–8510.

[6] Hansen, L. P., & J. Scheinkman (2009). Long-term risk: an operator approach. *Econometrica*, 77(1), 177–234.

[7] Huang, J. (2025). A probabilistic solution to high-dimensional continuous-time macro and finance models. CESifo Working Paper No. 10600.

[8] Lucas Jr, R. E. (1978). Asset prices in an exchange economy. *Econometrica*, 46(6), 1429–1445.